

Self-Healing Infrastructure with Terraform and AWS Lambda

Project Overview

This project demonstrates an automated self-healing infrastructure on AWS. Whenever an EC2 instance enters a stopped state, the system automatically detects the failure and restarts the instance without manual intervention. The entire infrastructure is provisioned and managed using Terraform.

Problem Statement

Manual monitoring and recovery of EC2 instances can lead to increased downtime and operational overhead. The goal of this project is to reduce downtime by implementing an automated recovery mechanism.

Architecture

EC2 Instance → CloudWatch EventBridge Rule → AWS Lambda → EC2 Restart

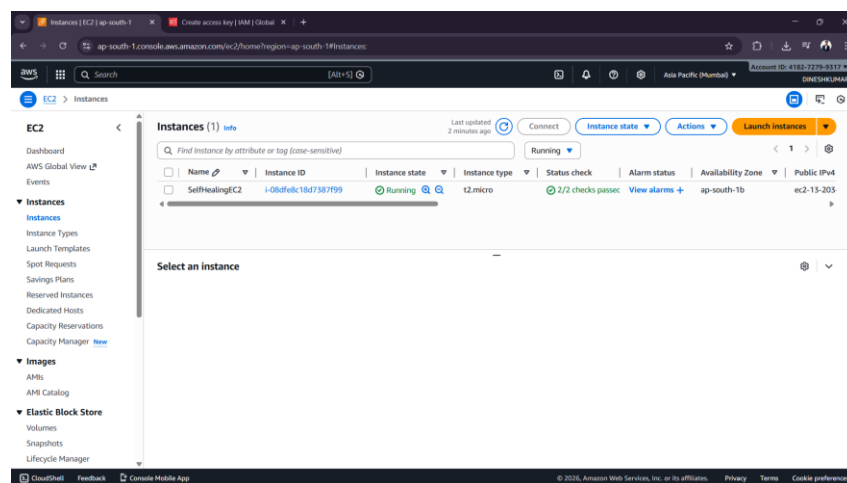
Tools & Technologies Used

- Amazon EC2
- AWS Lambda (Python)
- Amazon CloudWatch / EventBridge
- AWS IAM
- Terraform
- AWS CLI

Step-by-Step Implementation

Step 1: EC2 Instance Creation

An EC2 instance was created using Terraform. This instance acts as the target resource for the self-healing mechanism.



Command Used

```
DK@DK:~/self-healing-terraform$ aws configure
AWS Access Key ID [*****]: AKIAWCYYANJMR6D4RH64
AWS Secret Access Key [*****]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json
DK@DK:~/self-healing-terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.27.0...
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
DK@DK:~/self-healing-terraform$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions
are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.self_heal_ec2 will be created
+ resource "aws_instance" "self_heal_ec2" {
+   ami                    = "ami-0f5ee92e2d63afc18"
+   arn                    = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone       = (known after apply)
+   disable_api_stop        = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized           = (known after apply)
+   enable_primary_ipv6     = (known after apply)
+   force_destroy           = false
+   get_password_data       = false
+   host_id                 = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile    = (known after apply)
}
```

```
}
+ tags_all = {
+   + "Name" = "SelfHealingEC2"
+ }
+ tenancy = (known after apply)
+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ primary_network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

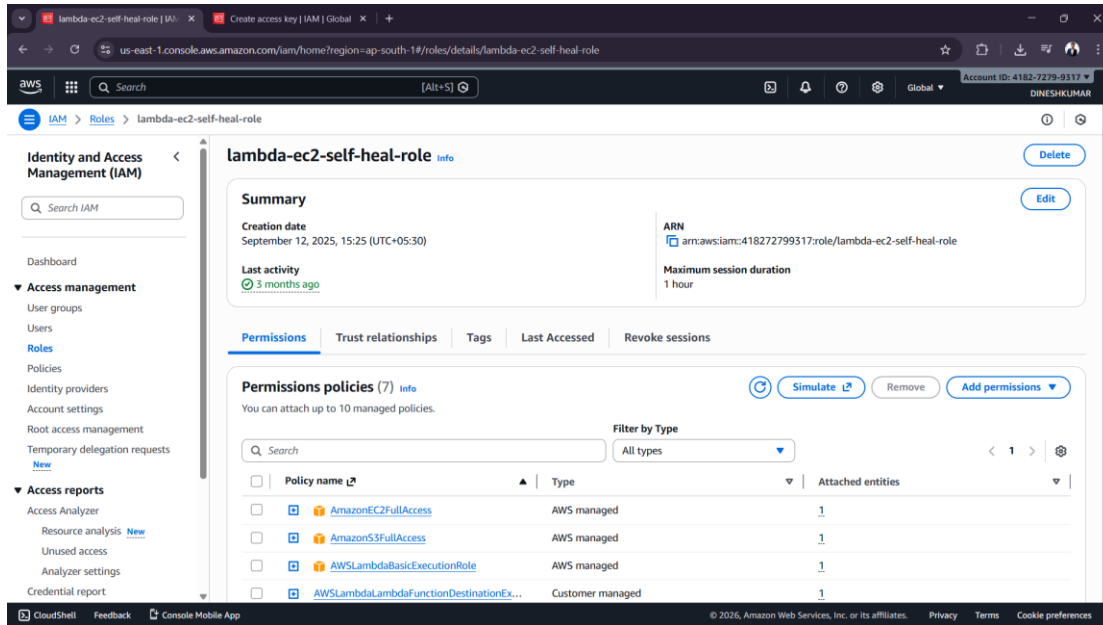
Enter a value: yes

aws_instance.self_heal_ec2: Creating...
aws_instance.self_heal_ec2: Still creating... [00m10s elapsed]
aws_instance.self_heal_ec2: Creation complete after 12s [id=i-08dfe8c18d7387f99]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
DK@DK:~/self-healing-terraform$
```

Step 2: IAM Role and Policy

An IAM role with least-privilege permissions was created to allow the Lambda function to describe and restart EC2 instances and write logs to CloudWatch.



Command Used

1st

```
DK@DK: ~/self-healing-terraform$ touch iam.tf
DK@DK: ~/self-healing-terraform$ vim iam.tf
DK@DK: ~/self-healing-terraform$ terraform apply
aws_instance.self_heal_ec2: Refreshing state... [id=i-08dfe8c18d7387f99]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_role.lambda_ec2_role will be created
+ resource "aws_iam_role" "lambda_ec2_role" {
+   arn                = (known after apply)
+   assume_role_policy = jsonencode(
+     {
+       Statement = [
+         {
+           Action      = "sts:AssumeRole"
+           Effect      = "Allow"
+           Principal   = {
+             Service = "lambda.amazonaws.com"
+           }
+         }
+       ]
+     }
+   )
+   version            = "2012-10-17"
+ }

# aws_iam_role_policy.lambda_ec2_policy will be created
+ resource "aws_iam_role_policy" "lambda_ec2_policy" {
+   id           = (known after apply)
+   name         = (known after apply)
+   name_prefix  = (known after apply)
+   policy       = jsonencode(
+     {
+       "Version": "2012-10-17",
+       "Statement": [
+         {
+           "Effect": "Allow",
+           "Action": [
+             "ec2:DescribeInstances",
+             "ec2:RestartInstances",
+             "logs:CreateLogGroup",
+             "logs:CreateLogStream",
+             "logs:PutLogEvents"
+           ],
+           "Resource": "*"
+         }
+       ]
+     }
+   )
+ }
```

2nd

```
DK@DK: ~/self-healing-terraform$ terraform apply
aws_iam_role.lambda_ec2_role: Refreshing state... [id=lambda-ec2-self-heal-role]
aws_instance.self_heal_ec2: Refreshing state... [id=i-0Bdfe8c18d73B7f99]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- update in-place

Terraform will perform the following actions:

# aws_iam_role.lambda_ec2_role will be updated in-place
- resource "aws_iam_role" "lambda_ec2_role" {
  - description      = "Allows Lambda to manage EC2 instances and write logs to CloudWatch." -> null
    id              = "lambda-ec2-self-heal-role"
    name            = "lambda-ec2-self-heal-role"
    tags            = {}
  }
  # (11 unchanged attributes hidden)

# aws_iam_role_policy.lambda_ec2_policy will be created
+ resource "aws_iam_role_policy" "lambda_ec2_policy" {
  + id              = (known after apply)
  + name            = (known after apply)
  + name_prefix     = (known after apply)
  + policy          = jsonencode(
    {
      + Statement = [
        + {
          + Action = [
            + "ec2:StartInstances",
            + "ec2:DescribeInstances",
          ]
          + Effect   = "Allow"
          + Resource = "*"
        },
        + {
          + Action = [
            + "logs:CreateLogGroup",
            + "logs:CreateLogStream",
            + "logs:PutLogEvents",
          ]
          + Effect   = "Allow"
          + Resource = "*"
        },
      ],
      + Version = "2012-10-17"
    }
  )
}
```

3rd

```
DK@DK: ~/self-healing-terraform$ terraform apply
# aws_iam_role_policy.lambda_ec2_policy will be created
+ resource "aws_iam_role_policy" "lambda_ec2_policy" {
  + id              = (known after apply)
  + name            = (known after apply)
  + name_prefix     = (known after apply)
  + policy          = jsonencode(
    {
      + Statement = [
        + {
          + Action = [
            + "ec2:StartInstances",
            + "ec2:DescribeInstances",
          ]
          + Effect   = "Allow"
          + Resource = "*"
        },
        + {
          + Action = [
            + "logs:CreateLogGroup",
            + "logs:CreateLogStream",
            + "logs:PutLogEvents",
          ]
          + Effect   = "Allow"
          + Resource = "*"
        },
      ],
      + Version = "2012-10-17"
    }
  )
  + role          = "lambda-ec2-self-heal-role"
}

Plan: 1 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_iam_role.lambda_ec2_role: Modifying... [id=lambda-ec2-self-heal-role]
aws_iam_role.lambda_ec2_role: Modifications complete after 2s [id=lambda-ec2-self-heal-role]
aws_iam_role_policy.lambda_ec2_policy: Creating...
aws_iam_role_policy.lambda_ec2_policy: Creation complete after 1s [id=lambda-ec2-self-heal-role:terraform-20260104065841455800000001]

Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
DK@DK:~/self-healing-terraform$
```

Step 3: Lambda Function

A Python-based AWS Lambda function was developed. It extracts the EC2 instance ID from the CloudWatch event payload and restarts the stopped instance automatically.

Command Used

```
DK@DK: ~/self-healing-terraform$ touch lambda_function.py
DK@DK:~/self-healing-terraform$ vim lambda_function.py
DK@DK:~/self-healing-terraform$ zip lambda.zip lambda_function.py
-bash: zip: command not found
DK@DK:~/self-healing-terraform$ sudo apt update
[sudo] password for DK:
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
Get:4 https://apt.releases.hashicorp.com trixie InRelease [12.9 kB]
Hit:5 http://ftp.debian.org/debian trixie-backports InRelease
Fetched 12.9 kB in 0s (52.2 kB/s)
22 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
 zip
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 22
Download size: 235 kB
Space needed: 642 kB / 1,018 GB available
Get:1 http://deb.debian.org/debian trixie/main amd64 zip amd64 3.0-15 [235 kB]
Fetched 235 kB in 0s (1,027 kB/s)
Selecting previously unselected package zip.
(Reading database ... 43985 files and directories currently installed.)
Preparing to unpack .../archives/zip_3.0-15_amd64.deb ...
Unpacking zip (3.0-15) ...
Setting up zip (3.0-15) ...
Processing triggers for man-db (2.13.1-1) ...
Scanning processes...

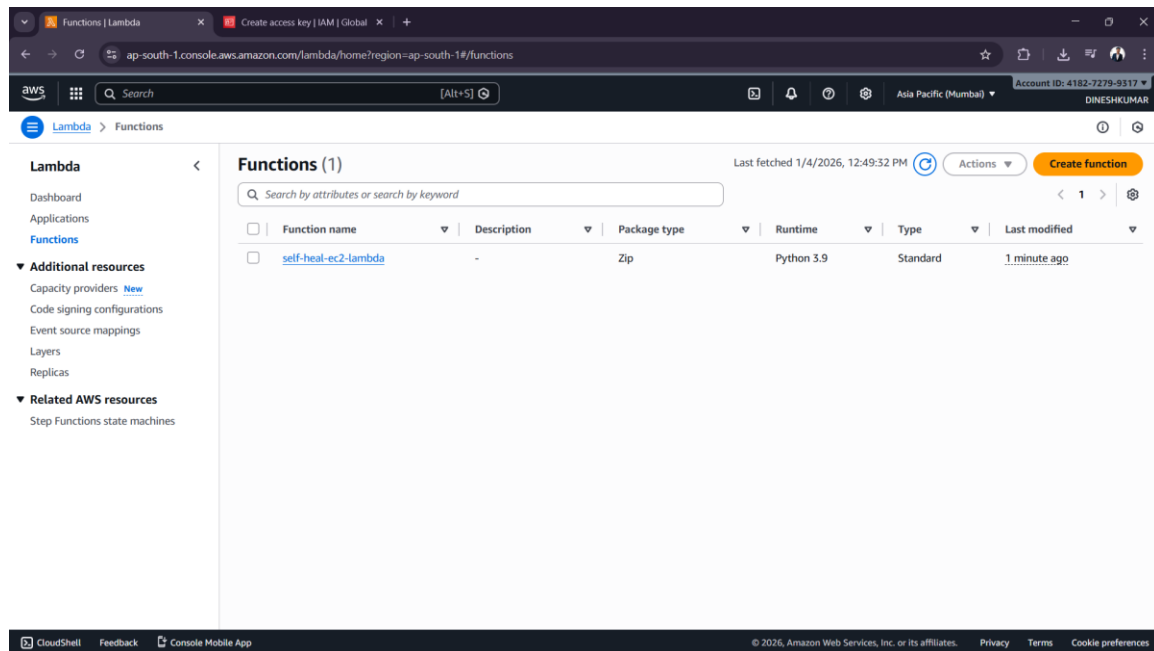
No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
DK@DK:~/self-healing-terraform$ zip lambda.zip lambda_function.py
adding: lambda_function.py (deflated 34%)
DK@DK:~/self-healing-terraform$ terraform apply
aws_instance.self_heal_ec2: Refreshing state... [id=i-08dfecl8d7387f99]
aws_iam_role.lambda_ec2_role: Refreshing state... [id=lambda-ec2-self-heal-role]
aws_iam_role_policy.lambda_ec2_policy: Refreshing state... [id=lambda-ec2-self-heal-role:terraform-26260194065841455800000001]

No changes. Your infrastructure matches the configuration.
```



Step 4: CloudWatch Event Rule

A CloudWatch EventBridge rule monitors EC2 instance state changes. When an instance enters the stopped state, the rule triggers the Lambda function.

Testing & Validation

The EC2 instance was manually stopped to validate the self-healing mechanism. CloudWatch successfully triggered the Lambda function, and the EC2 instance was restarted automatically.

Command Used

```
DK@DK: ~/self-healing-terraform$ terraform apply
aws_iam_role.lambda_ec2_role: Refreshing state... [id=lambda-ec2-self-heal-role]
aws_instance.self_heal_ec2: Refreshing state... [id=i-08dfe8c18d7387f99]
aws_iam_role_policy.lambda_ec2_policy: Refreshing state... [id=lambda-ec2-self-heal-role:terraform-20260104065841455800000001]
aws_lambda_function.self_heal_lambda: Refreshing state... [id=self-heal-ec2-lambda]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_cloudwatch_event_rule.ec2_stop_rule will be created
+ resource "aws_cloudwatch_event_rule" "ec2_stop_rule" {
+   arn                = (known after apply)
+   description        = "Trigger Lambda when EC2 stops"
+   event_bus_name     = "default"
+   event_pattern      = jsonencode(
+     {
+       detail          = {
+         state = [
+           "stopped",
+         ]
+       }
+       detail-type = [
+         "EC2 Instance State-change Notification",
+       ]
+       source        = [
+         "aws.ec2",
+       ]
+     }
+   )
+   force_destroy     = false
+   id                 = (known after apply)
+   name               = "ec2-stop-rule"
+   name_prefix       = (known after apply)
+   region             = "ap-south-1"
+   tags_all           = (known after apply)
+ }

# aws_cloudwatch_event_target.lambda_target will be created
+ resource "aws_cloudwatch_event_target" "lambda_target" {
+   arn                = "arn:aws:lambda:ap-south-1:418272799317:function:self-heal-ec2-lambda"
+   event_bus_name     = "default"
+ }

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_cloudwatch_event_rule.ec2_stop_rule: Creating...
aws_cloudwatch_event_rule.ec2_stop_rule: Creation complete after 0s [id=ec2-stop-rule]
aws_lambda_permission.allow_cloudwatch: Creating...
aws_lambda_permission.allow_cloudwatch: Creation complete after 0s [id=ec2-stop-rule-terraform-20260104095633982000000001]
aws_lambda_permission.allow_cloudwatch: Creation complete after 0s [id=AllowExecutionFromCloudWatch]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
DK@DK: ~/self-healing-terraform$
```

Instnce Stoppe

Successfully initiated stopping of i-08dfe8c18d7387f99

Instances (4) info

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☐	EKS-Bootstrap	i-Defaa91f1faec6f20	Stopped	t2.micro	-	View alarms +	ap-south-1b	-
☐	Jenkins-Agent	i-050e97ad4d2483f19	Stopped	t2.micro	-	View alarms +	ap-south-1b	-
☐	SelfHealingEC2	i-08dfe8c18d7387f99	Pending	t2.micro	-	View alarms +	ap-south-1b	ec2-3-110-1
☐	Jenkins-Master	i-0c9aa017a944d51de	Stopped	t3.micro	-	View alarms +	ap-south-1c	-

Select an instance

Automatically Run

Successfully initiated stopping of i-08dfe8c18d7387f99

Instances (1) info

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☐	SelfHealingEC2	i-08dfe8c18d7387f99	Running	t2.micro	Initializing	View alarms +	ap-south-1b	ec2-3-110-1

Select an instance

Outcome

- Automatic recovery of EC2 instances
- Reduced downtime
- No manual intervention required
- Fully automated using Infrastructure as Code

Conclusion

This project demonstrates a real-world DevOps use case of self-healing infrastructure using AWS services and Terraform. It highlights automation, scalability, and resilience in cloud environments.